

★ WEEK 9 · PART 2 · PRACTICE

The Kalman Gain, Worked by Hand

Turn the Dial Between Model and Data; Tune the Filter

How data quality and the Q/R ratio set the gain

Part 2's dial, with the arithmetic shown. We compute the gain across three data-quality regimes, watch the same surprise move the estimate differently, then find the steady-state gain and tune a filter from smooth to responsive.

2P

PART 2 · APPLICATIONS

The Dial Between Trust and Doubt

Case A: compute the gain for a clean quote, a normal print, and a noisy one - and watch the same surprise land differently. Case B: find the steady-state gain and tune a filter slow or fast with Q/R . Then a Claude Code lab.

What Part 2 Told Us

Now we turn the dial - with numbers

1

The gain balances model and data

K is state uncertainty over total uncertainty - a number between zero and one that says how much to trust the new print.

2

It shrinks the uncertainty

Every update lowers the variance P , so the filter grows more confident - and the gain adapts as P changes.

3

It settles to a steady state

After a while the gain stops changing, at a level set by the Q/R ratio. We compute that steady gain by hand.

Everything here is a calculator exercise: three noise levels, one steady-state formula, two tunings.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from the gain formula.
- Compute the gain for a few settings.
- Then read what it does to the estimate.

What you should watch

- How cleaner data raises the gain.
- How the same surprise lands differently.
- How Q/R tunes the filter's responsiveness.

The Gain Is a Dial

Between trusting the model and the data

The Kalman gain answers one question at every step: trust the model, or trust the new print? The answer depends on how noisy the data is. We fix the state uncertainty and compute the gain for a clean exchange quote, a normal print, and a noisy thin one - then watch the same surprise move the estimate by very different

The setup

- State uncertainty $P_- = 0.25$, fixed.
- The same innovation, 0.8, each time.
- Only the data noise R changes.

What we'll do by hand

- Compute the gain for three noise levels.
- Turn each gain into a correction.
- See clean data move the estimate most.

The Gain Formula

State uncertainty over total uncertainty

$$K = \frac{P^-}{P^- + R} = \frac{\text{state uncertainty}}{\text{state uncertainty} + \text{data noise}} \quad (\text{a number between 0 and 1})$$

Reading it

- The numerator is how unsure the model is.
- The denominator adds the data noise.
- So K is always between 0 and 1.

The two limits

- Tiny data noise: K near 1, trust the print.
- Huge data noise: K near 0, trust the model.
- Everything else sits on the dial between.

Three Data-Quality Regimes

Same state, different data noise

clean quote ($R = 0.05$) : $K = \frac{0.25}{0.30} = \mathbf{0.83}$ (trust the data)

normal ($R = 0.25$) : $K = \frac{0.25}{0.50} = 0.50$

noisy print ($R = 1.00$) : $K = \frac{0.25}{1.25} = \mathbf{0.20}$ (trust the model)

A clean quote ($R=0.05$) gives a gain of 0.83 - trust the data; a noisy print ($R=1.0$) gives 0.20 - trust the model. Same filter, opposite behavior.

Same Surprise, Different Correction

The gain scales the innovation

$$\text{correction} = K \times \nu : \quad 0.83(0.8) = 0.67, \quad 0.50(0.8) = 0.40, \quad 0.20(0.8) = 0.16$$

What happened

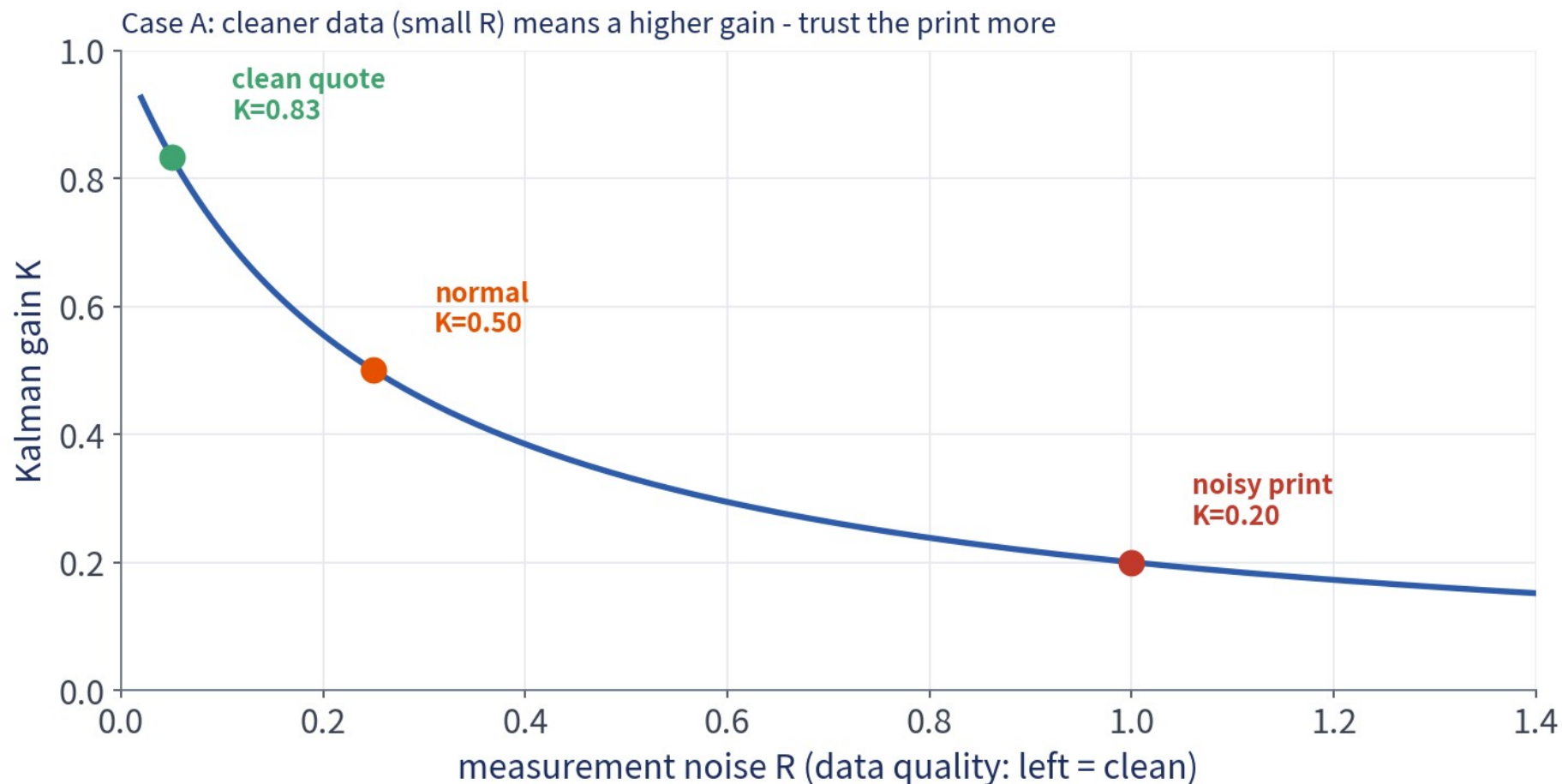
- The innovation was 0.8 in every case.
- The clean quote moved the estimate 0.67.
- The noisy print moved it only 0.16.

The lesson

- The surprise is the same size each time.
- The gain decides how much of it to believe.
- Data quality, not the surprise, sets the response.

The Gain Dial

As the data gets noisier (R grows to the right), the gain falls: the filter trusts each print less and leans on the model more. The clean quote, the normal print, and the noisy print sit at three points along the same smooth dial



Why This Is Automatic

The filter reads data quality for you

Without the gain

- You'd pick one blend of model and data.
- It would be wrong when data quality changes.
- A thin-market print would fool you.

With the gain

- Noisy prints are down-weighted automatically.
- Clean quotes get taken more seriously.
- The dial turns itself, every step.

Tuning the Filter

From smooth and laggy to fast and jumpy

Run the filter a while and the gain stops changing - it reaches a steady state set by the ratio of process noise Q to measurement noise R . That ratio is your tuning knob: a small Q gives a smooth, laggy filter; a large Q gives a responsive, noisier one. We compute the steady-state gain for both by hand

The idea

- The variance settles, so the gain settles.
- The steady gain depends only on Q and R .
- Q/R is the responsiveness knob.

What we'll do by hand

- Use the steady-state gain formula.
- Compute it for a slow and a fast filter.
- See the trade-off between smooth and quick.

Why the Gain Settles

Predict grows P , update shrinks it

The tug of war

- Predict adds Q - uncertainty grows.
- Update multiplies by $(1-K)$ - it shrinks.
- The two forces reach a balance.

At the balance

- The variance stops changing: steady state.
- So the gain stops changing too.
- It depends only on Q and R , not the data.

The Steady-State Gain

When the dial stops turning

$$p = \frac{Q + \sqrt{Q^2 + 4QR}}{2}, \quad K_{\infty} = \frac{p}{p+R} \quad (\text{the gain stops changing})$$

Solve the balance for the steady predicted variance p , then form the gain. It no longer depends on the data - only on how fast the state drifts versus how noisy the prints are.

Slow vs Fast

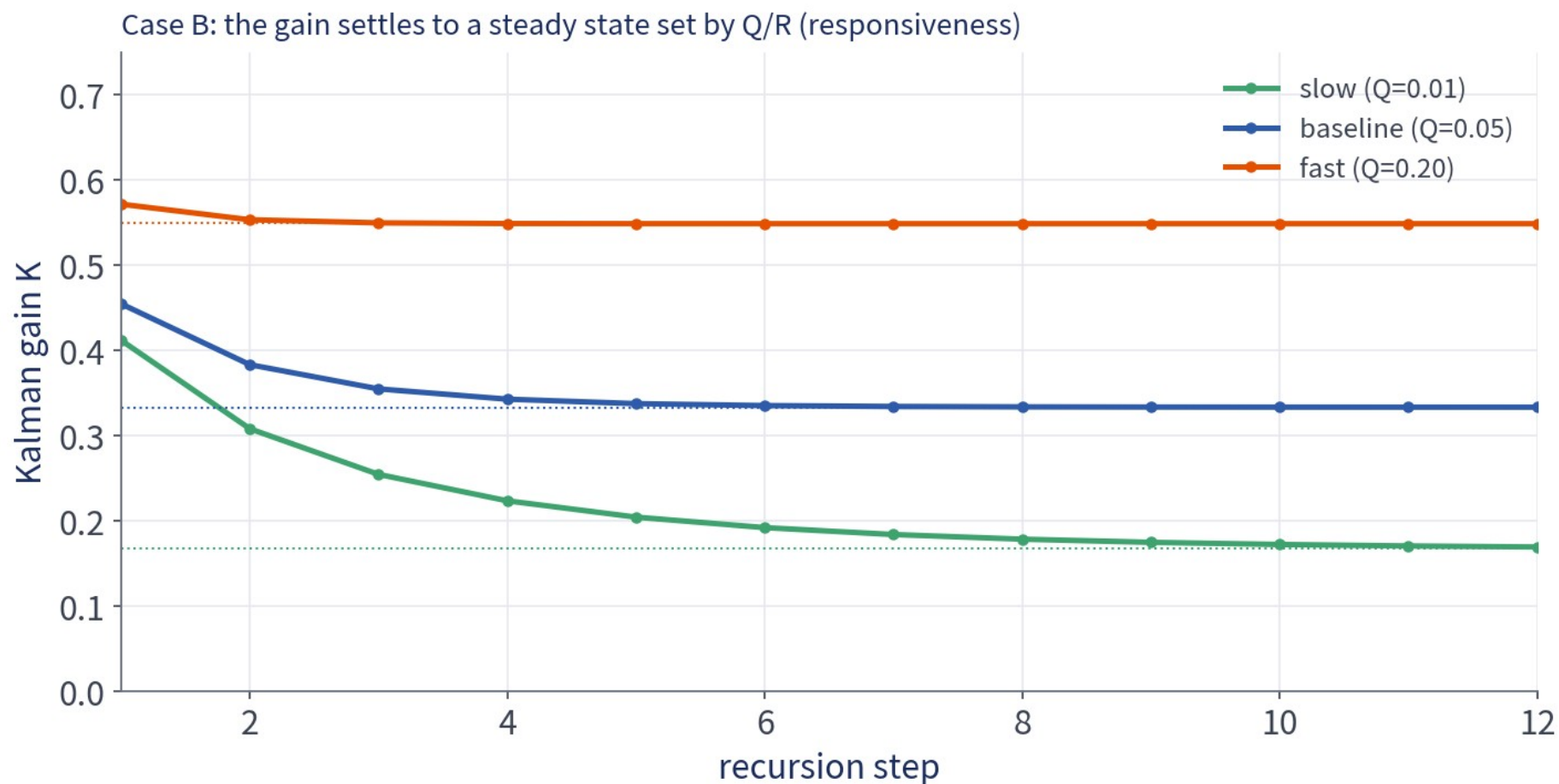
Two tunings, two steady gains

$$\begin{aligned} \text{slow } (Q = 0.01) : p = 0.060, \quad K_{\infty} &= \frac{0.060}{0.360} = \mathbf{0.17} \quad (\text{smooth, laggy}) \\ \text{fast } (Q = 0.20) : p = 0.365, \quad K_{\infty} &= \frac{0.365}{0.665} = \mathbf{0.55} \quad (\text{responsive, noisy}) \end{aligned}$$

A small Q settles to a gain of 0.17 - smooth but slow to react; a large Q settles to 0.55 - quick to follow moves but noisier. You pick the trade-off.

The Gain Settles to Steady State

From the same start, each filter's gain converges to its own steady level: the slow filter to 0.17, the baseline to 0.33, the fast one to 0.55. The Q/R ratio, chosen in advance, decides where the dial finally rests.



Responsive or Smooth?

A design choice you control

A fast filter (high Q)

- Catches regime shifts quickly.
- But passes more noise into the estimate.
- Good when the state really moves.

A slow filter (low Q)

- Gives a smooth, stable estimate.
- But lags when the state actually changes.
- Good when the state is nearly constant.

The Common Pattern

The gain is the whole story

Both cases are about one number. Moment to moment, the gain reads data quality and decides how much of each surprise to believe. Over the long run, the Q/R ratio you chose fixes where that dial settles. Master the gain, and you have mastered the filter's judgment

Short run

- The gain rises with cleaner data.
- It scales each surprise into a correction.
- It turns itself, step by step.

Long run

- The variance and gain settle to steady state.
- Q/R sets responsiveness versus smoothness.
- You tune it before you ever see the data.

Reproduce the Dial and the Tuning

Direct Claude Code to check your arithmetic

GOAL Verify the gain across regimes and the steady-state tuning, then plot both.

[INPUT] Prompt · Claude Code

```
Write and run part2_practice.py that:  
CASE A - the dial:  
  * with  $P_- = 0.25$ , print  $K = P_- / (P_- + R)$  for  
     $R = 0.05, 0.25, 1.0$ ; print  $K * 0.8$  each time;  
CASE B - steady state:  
  * with  $R = 0.30$ , compute  $p = (Q + \sqrt{Q^2 + 4QR}) / 2$   
    and  $K = p / (p + R)$  for  $Q = 0.01, 0.05, 0.20$ ;  
  * run the recursion and confirm  $K$  converges there.  
Then plot  $K$  vs  $R$  (the dial) and  $K$  vs step (settling).  
Match every printed number to the slides.
```

Check against the slides

- $K = 0.83, 0.50, 0.20$.
- corrections $0.67, 0.40, 0.16$.
- steady $K = 0.17, 0.33, 0.55$.
- gain converges to those.

Deliverable: part2_practice.py whose printed numbers match the dial and the steady-state tunings.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What happens to the gain as data gets noisier?

It falls - trust the model more

Q2 What sets the steady-state gain?

The Q/R ratio, not the data

Q3 What does a large Q give you?

A fast, responsive, noisier filter

All three come from the gain dial and the steady-state tuning.

Three Pitfalls to Avoid

Where the gain trips people up

Common mistakes

- "A high gain is always better."
- "Set Q and R once and forget the ratio."
- "The steady gain depends on the data."

The reality

- A high gain is jumpy - it passes noise through.
- Q/R is a design choice: smooth vs responsive.
- The steady gain depends only on Q and R.

Two Calculations to Remember

Part 2 applications in one screen

1

The gain is a dial on data quality

With state uncertainty fixed at 0.25, a clean quote ($R=0.05$) gave a gain of 0.83 and a noisy print ($R=1.0$) gave 0.20 - so the same 0.8 surprise moved the estimate 0.67 versus 0.16. The filter reads data quality for you.

2

The gain settles, and Q/R tunes it

The steady-state gain solves a simple balance: a slow filter ($Q=0.01$) settled to 0.17, smooth but laggy; a fast one ($Q=0.20$) to 0.55, responsive but noisy. You choose the trade-off with Q/R.

3

Master the gain, master the filter

Short run, the gain weighs each surprise; long run, Q/R fixes where the dial rests. Everything the Kalman filter does flows through this one number.

Next — Part 3 practice: the pairs trade and nowcasting, where the filter meets real markets.